Handoff: hunting PostgreSQL bugs with the CS-5187 fuzzer

Everything done so far, the state of every lead, and exactly how to continue (written 17 Oct 2026)

0. Read this first (for the person and for their Claude)

Mission. Find a previously unknown PostgreSQL bug (crash, assertion failure, wrong result, or recovery failure), reduce it, report it to the PostgreSQL developers, send a patch, and get a reply or a fix commit. That upstream acknowledgement is what the course gives top marks for.

How PostgreSQL accepts work (important, easy to get wrong). PostgreSQL does **not** take GitHub pull requests. `github.com/postgres/postgres` is a read-only mirror. The "PR" is:

1. a bug report to **pgsql-bugs** (web form at postgresql.org/account/submitbug, needs a free community account) or a mail to `pgsql-bugs@lists.postgresql.org`, and
2. a **patch file** (`git format-patch`) attached to that thread or posted on **pgsql-hackers**. A committer reviews it on the list; if it is good they commit it with "Reported-by: (your name)". The evidence for the grade is the thread URL, the bug number, and the fix commit.

Status in one table.

Lead	What	State
BUG-01	`JSON_TABLE` never assigns collations to its context item and `PASSING` arguments: assertion crash on assert builds, **wrong result** and spurious "could not determine which collation" errors on the 18.6 release	**Real. Reproduced. Root-caused. Patch + regression tests written, full regression suite 239/239 passes. NOT YET FILED.** This is the thing to file first.
Candidate 2	Crash-recovery failure: after a crash, restart died with `FATAL: could not create file "base/5/165792": File exists` (an unlogged sequence's init fork)	**Seen once, evidence saved, NOT reproduced yet.** Unfiled.
False positives	Our differential oracle reported `ARRAY(SELECT ...)` order differences	Fixed in our code. Not PostgreSQL bugs.

Do these in order: (1) check that BUG-01 is not already reported/fixed upstream (the sandbox could not reach postgresql.org, so this was **not** done); (2) file BUG-01 with the patch; (3) keep fuzzing and try to reproduce candidate 2.

What the earlier session could not do: reach `www.postgresql.org`, `postgr.es`, `commitfest.postgresql.org` (blocked by the sandbox network policy). So **nobody has searched the mailing-list archives for duplicates yet.** On a normal laptop that is a 10-minute job and it is mandatory before filing.

1. The project in brief

CS-5187 term project A3: *LLM-guided grammar-aware fuzzing of PostgreSQL* (Venu, Bhuvan, Gowtham). The repository is `https://github.com/v28k/cs5187-term-project` (branch `main`). Its own documents are in `docs/learn/` and `progressmeet1/`; the useful summary:

- `src/pgfuzz/` is a Python fuzzer. A **generator** produces test cases (lists of SQL statements); a **PostgresExecutor** runs them on a server in a savepoint and rolls back; an **oracle** classifies the outcome (crash, `TRAP:` assertion, internal error `XX000`, wrong result) and deduplicates by signature.
- The **grammar arm** (`src/grammar/postgres.pgg`, typed and scope-aware) generates almost only valid SQL (966/966 accepted in the team's last sample).
- The **differential plan oracle** re-runs deterministic `SELECT`s under up to 2 of 17 planner settings (hash joins off, forced parallel, ...) and compares result multisets.
- The server under test is PostgreSQL `master` (`20devel`) built with `--enable-cassert` so internal checks print `TRAP: failed Assert(...)`.
- Team plan/dates: meeting 2 is 26-30 Oct, meeting 3 is 2-6 Nov, presentation 12 Nov, paper and repo due 13 Nov 2026.

2. Repository layout added by the bug-hunt sessions

```
docs/bughunt/
  HANDOFF.md                  this document (PDF copy: HANDOFF.pdf)
  LOG.md                      running log: what was read, run, found
  BUG-01-json_table-collation.md   full report + draft email text
  0001-Assign-collations-...patch  the patch, git format-patch, against master 26bd08fb
  evidence/                   bug01-original-fuzzer-query.sql, bug01-trap-backtrace.txt,
                              cand2-recovery-failure-log.txt, cand2-pg_controldata.txt,
                              cand2-waldump-relfilenode-165792.txt
scripts/
  setup_env.sh                rebuild master (cassert) + REL_18 from source
  cloud_hunt.sh               start the 6-hour grammar campaign on ports 5432/5433
  joinfuzz_loop.sh            endless cross-version join fuzzer v1
  joinfuzz2_loop.sh           endless cross-version join/aggregate fuzzer v2
src/configs/cloud_hunt.yaml   hunt.yaml with Linux log paths, 6 h per trial
src/configs/cloud_local.yaml  local.yaml with Linux log paths (selftest, replay)
src/xversion/joinfuzz.py      v1: nested joins, LATERAL, sublinks, (whole-row) IS NULL
src/xversion/joinfuzz2.py     v2: bigger typed tables, random GUCs on master, GROUP BY over joins
src/pgfuzz/eval/differential.py   eligible(): now excludes ARRAY(SELECT ...) without ORDER BY
src/tests/test_differential.py    7 tests for that
```

`results/` and `src/results/` are git-ignored (run output). Everything that matters for the grade is committed and pushed (last pushed commit: `0b6093f`, plus the commit that adds this document).

3. What was done, in order

1. **Read the project** (README, all `docs/learn/*.md`, speaker notes, configs, harness scripts). Nothing in `known_bugs/known_bugs.yaml` yet (empty list).
2. **Built two servers from source** (4-core cloud container, about 20 min for master):
   - master `26bd08fb` (16 Oct 2026), `--enable-cassert --enable-debug, -O1;`

    - `REL_18_STABLE` (18.6, commit `92d47d5`) as a plain reference. Servers cannot run as root, so an unprivileged user `pg` runs them (`harness/local/pg_local.sh` with `PG_BIN=/root/pgfuzz/install/master/bin ROOT=/home/pg/pgfuzz`): ports 5432 and 5433 for the campaign, 5434 manual master, 5435 release reference.
3. **Verified the tool:** `pytest` 84/84, `selftest` 10/10 on the new build.
4. **Read recent master source for bug-prone new code** (list in section 5).
5. **Ran the grammar campaign** (`configs/cloud_hunt.yaml`, two trials). In ~11-18 minutes it produced the first real find (BUG-01) plus false positives. The campaign was later stopped (container restart); trials only ran ~11 and ~18 minutes, so this is a *tiny* fraction of a planned 6 h run.
6. **Wrote cross-version differential fuzzers** (`xversion/joinfuzz*.py`): run the same random query on master and on 18.6; any difference in result multiset, or an error/crash on only one side, is saved as a replayable `.sql`. v1 ran about 80,000 queries with zero mismatches. v2 ran briefly; its few "crash" files were caused by our own server restarts and were discarded.
7. **BUG-01** found, minimised, root-caused, fixed, tested (section 4).
8. **Candidate 2** found when server pg-0 failed to restart (section 6).
9. Everything committed and pushed to GitHub as evidence.

4. BUG-01 in full (the lead to file first)

One sentence. `transformJsonTable()` in `src/backend/parser/parse_jsontable.c` calls `transformExpr()` on the JSON_TABLE context item and PASSING arguments but never `assign_expr_collations()`, so those expressions keep no collation.

Symptoms (all verified):

Build	Query	Result
master, cassert	`SELECT * FROM JSON_TABLE(jsonb_build_object('k', ROW(1,2) < ROW(3,4)), '$' COLUMNS (a bool PATH '$.k'));`	backend aborts: `TRAP: failed Assert("list_length(rcexpr->inputcollids) == nopers"), File: "execExpr.c", Line: 2078`
18.6 release	same	returns **f** instead of `t` (silent wrong result)
18.6 release	`... jsonb_build_object('k', 'a' < 'b' COLLATE "C") ...` or the same in `PASSING`	`ERROR: could not determine which collation to use for string comparison`

Why: a `RowCompareExpr` with an empty `inputcollids` list makes the executor compile zero column comparisons. XMLTABLE already does it right (`transformRangeTableFunc` in `parse_clause.c` runs `assign_expr_collations`). JSON_TABLE exists in 17, 18, 19 beta and master; REL_17 has the same omission.

The fix (one line, plus tests):

```
            tf->docexpr = transformExpr(pstate, (Node *) jfe, EXPR_KIND_FROM_FUNCTION);
        +   assign_expr_collations(pstate, tf->docexpr);
```

It must come before `tf->passingvalexprs = copyObject(je->passing_values);` so the copies are collated too (it does).

Verification done: new tests in `src/test/regress/sql/sqljson_jsontable.sql` fail on unpatched 18.6 and pass patched; whole `make check` = **All 239 tests passed**.

Where the work is: the patched tree is `~/pgfuzz/src/postgres-master` (commit `72620fa2`, local only). The exported patch is committed in `docs/bughunt/`. Apply on a fresh master checkout with `git am` (or `git apply`); if it does not apply cleanly, redo the 1-line change and the tests by hand, then re-run `make check`.

Honest caveats before filing:

- Not searched upstream yet (see section 0). Search pgsql-bugs/pgsql-hackers for "JSON_TABLE collation", "inputcollids", "RowCompareExpr JSON_TABLE".
- The assert crash happens only on assert-enabled builds; on release builds it is the wrong result and the collation error. Say so in the report.
- A committer may prefer the call placed elsewhere (e.g. after the PASSING copy or inside `transformJsonTableColumns`). Be ready to adjust.

Filing steps:

1. Create a community account, then use postgresql.org/account/submitbug. Paste the "Draft report text" from `BUG-01-json_table-collation.md` (version 18.6, OS, the reproducer, one-line cause).
2. In the same thread (or on pgsql-hackers) attach the `.patch`. Subject style: `[PATCH] JSON_TABLE: assign collations to context item and PASSING args`.
3. Answer every reviewer question quickly; re-test requested changes against `make check`.
4. Record the bug number, thread URL and commit in `docs/bughunt/LOG.md`.

5. Where the source reading went (and what was ruled out)

Master has been churning; many fix commits in the last two weeks mean sibling bugs are likely. Read so far:

- **Outer-join to anti-join reduction** (`reduce_outer_joins_pass2`, `forced_null_var_is_nonnullable`, `find_safe_quals`, `strip_redundant_nullability_quals` in `prepjointree.c` / `clauses.c`; commits a812cf64, 2ce5a57b, 02119d6c, f863be3e, 31c002aa, ad4b80b6). Checked system columns, NOT VALID not-null, virtual generated columns, inheritance parents, whole-row Vars. **No hole found by reading.**
- **Eager aggregation** (`setup_eager_aggregation`, `init_grouping_targets`, `get_expression_sortgroupref` in `relnode.c`/`initsplan.c`): equal-image checks are present; recent fixes d764d61a, ffeda042, 3a08a2a8, bd94845e. No hole found.
- **Virtual generated columns + NOT IN anti-join** (fix d8444163 only covers *partitioned* tables): worth probing other table shapes (inheritance, views).
- Recent-feature areas not yet read: join removal on the jointree (2ebf25e7 and ~6 follow-ups), self-join elimination, `JSON_TABLE PLAN` clause (86ab7f4c), `GRAPH_TABLE` (reverted), RI fast-path code (a dozen fixes, late August to October), REPACK (CONCURRENTLY), data checksums launcher.

Useful trick: `git log --since=... --grep=Fix` on `src/backend/optimizer`, `executor`, `parser`, `utils/adt`. A feature with many "Fix …" follow-ups usually hides one more bug.

6. Candidate 2: crash-recovery failure (unreproduced)

What happened. While the campaign was running, server pg-0 was killed by an assertion (BUG-01). It restarted into crash recovery and **failed**, permanently:

```
LOG:  redo starts at 0/1DEAA0D0
LOG:  invalid record length at 0/1E59B140: expected at least 24, got 0
LOG:  redo done at 0/1E59B0D8
FATAL:  could not create file "base/5/165792": File exists
LOG:  startup process (PID 29718) exited with exit code 1
```

Findings from `docs/bughunt/evidence/`: `base/5/165792` is an empty file and `165792_init` is 8192 bytes; the WAL (read with `pg_waldump`) contains, for relfilenode 165792, `LOCK`, `CREATE base/5/165792_init`, and a `Sequence LOG rel .../165792 fork init` record, i.e. an **unlogged sequence** created in a transaction that was open at the crash. The failing step is `ResetUnloggedRelations(UNLOGGED_RELATION_INIT)` in `xlog.c` (~line 6629) → `copy_file()` in `copydir.c` opening the main fork with `O_CREAT | O_EXCL`. Relevant new master code: `XLogFlushBufferForRedoIfInit()` (commit 8e684ce1, 30 Jun 2026). Hypothesis (unproven): the main fork file already exists from an earlier pass, so the exclusive create fails, because the CLEANUP pass only removes main forks whose init fork was present *before* WAL replay, while this init fork was created *by* WAL replay (the file was created by the first, interrupted startup attempt and a second attempt then trips on it).

Reproduction attempts (all negative): a committed unlogged sequence plus immediate stop; an uncommitted one plus immediate stop; both recover fine. The original sequence of events was a **chain** of crashes on a server with `restart_after_crash=on`, mid-transaction creation of an unlogged table/sequence, repeated within seconds.

Next steps:

1. Keep the evidence; do not delete `/home/pg/pgfuzz/pg-0-recovery-failure-snapshot` if the machine is still alive (247 MB data dir, includes WAL and log). It is on the cloud container only; the small text extracts are in git.
2. Retry in a *controlled* way: create an unlogged sequence and table inside an open transaction, `pg_ctl stop -m immediate`, restart, and *interrupt the startup* during recovery (second crash during recovery, e.g. `kill -9` of the startup process, or an injected `FATAL` after replay) so that `ResetUnloggedRelations(INIT)` has half-run; then restart again and see whether `copy_file` hits `File exists`. Also try on `REL_18_STABLE`: if it fails there too it is an older bug.
3. Read `ResetUnloggedRelationsInDbspaceDir` and `ResetUnloggedRelations` call sites in `xlog.c` (CLEANUP at ~6479 before replay, INIT at ~6629 after replay) for the ordering assumption.
4. Only report if it reproduces from a clean data directory with exact steps.

7. Restarting everything on a new machine

```
git clone https://github.com/v28k/cs5187-term-project && cd cs5187-term-project
bash scripts/setup_env.sh          # builds master(cassert) + REL_18 into ~/pgfuzz  (~25 min)
```

Then (as a **non-root** user; on root-only containers create user `pg` and run the servers as it, giving it read access to the install dir):

```
        export PG_BIN=$HOME/pgfuzz/install/master/bin ROOT=$HOME/pgfuzz
        bash harness/local/pg_local.sh init 3 && bash harness/local/pg_local.sh start 3   # 5432-5434
        # reference release server on 5435: initdb with the REL_18 binaries, port = 5435
        cd src && pip install -r requirements.txt && python3 -m pytest -q   # expect 91 passed
```

Edit `server_log:` in `configs/cloud_*.yaml` to your data-dir path, then:

```
        python3 -m pgfuzz selftest configs/cloud_local.yaml          # 10/10
        bash scripts/cloud_hunt.sh                                    # grammar hunt, results in
src/results/cloud_hunt/
        bash scripts/joinfuzz2_loop.sh 1                              # cross-version fuzzer, saves
src/results/joinfuzz2/*.sql
```

A bug candidate is in `src/results/cloud_hunt/grammar/trial-NN/failures/<sig>/` (`failure.json`, `testcase.sql`, `repro-K.sql`). **Warning:** `testcase.sql` may be a *later* duplicate rather than the case that first triggered the signature (this happened). The reliable source is `events.jsonl` in the trial directory (look for the event whose `outcome` is the bug and `sig_new: true`), plus the `TRAP:` line and backtrace in the server's `postgresql.log`.

Triage recipe for any candidate:

1. Replay it alone on a fresh server: `python3 -m pgfuzz replay configs/cloud_local.yaml X.sql --repeat 3`.
2. Reduce by hand (delete clauses until it stops failing); keep a query under ~10 lines.
3. Run the reduced query on the release server (18.6) and on `master`; note which fail.
4. Read the stack/assert location in the source and find the cause.
5. Search upstream archives for duplicates *before* writing anything.
6. Write the fix, add a regression test, run `make check` (and the relevant `make installcheck-tests TESTS='...'`).
7. `git format-patch -1`, then file as in section 4.

8. Planned next steps (priority order)

1. **File BUG-01** (section 4). Highest value, nearly done.
2. **Resume the grammar hunt** for the planned 4-6 h per trial. In 18 minutes it already found a real bug, so hours should find more. Rebuild master first (the patched tree fixes BUG-01 and stops it from repeatedly killing the server; or just reuse the unpatched build and treat that one signature as known).
3. **Reproduce candidate 2** (section 6) and file if real.
4. **Run `joinfuzz2` for hours**, and extend it where it found nothing: add `MERGE` / `UPDATE ... FROM` / `DELETE ... USING` shapes, `RETURNING OLD/NEW`, partitioned targets with generated columns, SQL/JSON functions, `TABLESAMPLE`, recursive CTEs.
5. **Hunt around BUG-01's pattern:** any other node type the parser builds for a *FROM-clause function* that skips `assign_expr_collations`. Candidates to test: `JSON_TABLE` column `DEFAULT` expressions and `NESTED PATH` columns, `XMLTABLE` columns, `ROWS FROM(...)`, `TABLESAMPLE` arguments, function arguments in `LATERAL` calls. A quick probe of 24 other expression contexts (CHECK, DEFAULT, generated columns, policies, triggers, MERGE, ON CONFLICT, RETURNING, window frame offsets, publication row filters, …) found no further crash, but JSON_TABLE `PLAN/NESTED PATH` columns were only partly probed.

6. **Metamorphic oracles from the project plan** (TLP, NoREC, equivalent-expression rewrites, `docs/learn/02_future_plan.md` section 3): they find wrong-result bugs that planner-setting differences cannot see. High value for the course too.
7. **Fix false positives in our oracle** whenever one appears (we fixed `ARRAY(subquery)`); each one wastes triage time. Candidates still seen in the log but not yet analysed: `d437283ebe37` (`parallel` variant, `ARRAY(SELECT ... GROUP BY CUBE ...)` inside `jsonb_build_object`) and `67b3ae706302` (`no_hashagg`, `ARRAY(SELECT ... GROUP BY ...)`) are almost certainly the same unordered-ARRAY issue, but confirm by running them with `ORDER BY` inside the `ARRAY(...)`.
8. **Write up for the course:** copy final evidence into `paper/data/`, fill the report sections (motivating example = BUG-01 with real numbers), follow `paper/STYLE.md` (every number from a real run, no invented citations).

9. Pitfalls and lessons (read before running anything)

- PostgreSQL's `initdb` and servers **refuse to run as root**. Use a normal user.
- Use `127.0.0.1` not `localhost` in connection strings (IPv6 delays).
- The sandbox blocked `postgresql.org`. On a normal machine use the archives (postgresql.org/list/pgsql-bugs/, pgsql-hackers) and the PG19 open items wiki.
- `pkill -f pattern` from a shell whose own command line contains `pattern` kills that shell; use `pkill -f "[p]attern"` in a separate command.
- A crash-heavy test (many aborted backends in a row) makes the fuzzers record spurious "crash" files and can wreck the data directory. Do such experiments on a throwaway cluster on its own port, never on the campaign servers.
- Postgres on `master` changes daily; rebuild and re-test a bug against the newest `master` before filing (`git pull`, `make`, re-run the reproducer).
- The project's own `known_bugs/known_bugs.yaml` needs an entry only if a bug's public source (thread URL) exists; add BUG-01 after filing.
- Be honest in the report: say which results are from real runs, and that candidate 2 is unreproduced until it is.

10. Prompt to give the friend's Claude

> You are continuing a PostgreSQL bug hunt. Read `docs/bughunt/HANDOFF.md` fully, then `docs/bughunt/LOG.md` and `docs/bughunt/BUG-01-json_table-collation.md` in the repository https://github.com/v28k/cs5187-term-project. First task: confirm BUG-01 is not already reported upstream (search the pgsql-bugs and pgsql-hackers archives and the git log of master), rebase the patch onto current master, run `make check`, and help me file the bug report and patch to the PostgreSQL mailing lists. Second task: set up the environment with `scripts/setup_env.sh`, resume the fuzzing campaigns, and try to reproduce candidate 2 (section 6). Do not invent results, citations or reproducers; every claim must come from a run.